

SuperPlots in Python (Matplotlib + Seaborn)

HCI/HCA Course

NanoCell Interactions Lab

2026-07-22

Table of contents

0.1	Step 1: Create the Pixi environment	2
0.2	Step 2: Install the environment	3
0.3	Step 3: Open the project in your favorite app	4
0.4	Optional: open an activated shell	4
0.5	Verify the installation	4
0.6	Step 2: Organize your data	5
0.7	Step 3: Load and sanity-check	6
0.8	Step 4: Compute replicate means (for stats & big markers)	6
0.9	Step 5: Choose and run the right statistical test	6
0.10	Step 6: Make the SuperPlot	8
0.11	Step 7: Save publication-ready figures	10
0.12	Step 8: Wrap it into a reusable function	10
0.13	Step 9: Common variants	12
0.14	Step 10: Reproducibility tips	13

This tutorial shows how to make **SuperPlots**, figures that communicate *both* single-cell variability and experiment-level reproducibility.

Core idea: plot all cell-level measurements (small points), overlay replicate means as large, outlined markers colored by replicate, and run stats on the replicate means (not on single cells).

To illustrate how important transparent data visualization has become in current publishing standards, consider the following feedback we received after submitting a manuscript to the *Journal of Cell Science*:

For best practice and transparency, and to allow better assessment of the quality of the data and whether the data support the conclusions, we strongly recommend that you:

1. Use graphs that allow the reader to see the true data spread (i.e. box and whisker plots, super plots, etc. See [PMID: 32346721](#) for more information.). If using box-and-whisker plots, please also state what the whiskers represent. See [here](#) for helpful information.
2. Use appropriate statistics with the sample size representing biological replicates rather than technical replicates
3. Provide, if using western blots, one additional supplementary figure (as a single PDF or TIFF file). It will not contribute towards your supplementary figure limit. If you provided this at submission and named it 'Blot transparency', please now number this supplementary figure and combine it with the rest of the supplementary material pdf. Please ensure that you cite this figure in your Materials & Methods section, ideally in the western blots paragraph.

0.1 Step 1: Create the Pixi environment

This tutorial uses [Pixi](#) to create a reproducible Python environment. Pixi installs Python and all required packages from a single configuration file, so you do not need to create or activate a Conda environment manually. If you are using the Pixi environment provided for the HCI course, you do not need to follow the environment-creation steps below. The course environment already includes all packages required for this tutorial. If you are working outside the course environment or are testing the SuperPlots tutorial independently, follow the instructions below.

Create a new folder for the tutorial and enter it:

```
mkdir superplots-tutorial
cd superplots-tutorial
```

Create a file named `pixi.toml` with the following content:

```
[workspace]
name = "superplots-tutorial"
version = "0.1.0"
description = "Minimal environment for plotting complete datasets and SuperPlots"
channels = ["conda-forge"]
platforms = ["linux-64", "osx-64", "osx-arm64", "win-64"]

[dependencies]
python = "≥3.10,<3.13"
pandas = "≥2.0"
numpy = "≥1.24"
matplotlib = "≥3.7"
seaborn = "≥0.12"
scipy = "≥1.11"
ipykernel = "≥6.0"
jupyterlab = "≥4.0"

[tasks]
notebook = "jupyter lab"
```

Your folder should now contain:

```
superplots-tutorial/
└─ pixi.toml
```

0.2 Step 2: Install the environment

From the folder containing `pixi.toml`, run:

```
pixi install
```

Pixi will create the environment and generate a `pixi.lock` file containing the resolved package versions. Keep both `pixi.toml` and `pixi.lock` with the tutorial to make the environment reproducible.

0.3 Step 3: Open the project in your favorite app

0.3.1 Step 3a: Open the project in VSCode

Open the tutorial folder in VSCode:

```
code .
```

0.3.2 3b: Open the project in JupyterLab

Run:

```
pixi run notebook
```

This command starts JupyterLab inside the Pixi environment with all required packages available.

Alternatively, you can run JupyterLab directly:

```
pixi run jupyter lab
```

0.4 Optional: open an activated shell

It is not necessary to activate the environment before running the tutorial. Pixi can execute commands directly with `pixi run`.

If you prefer to work inside an activated shell, use:

```
pixi shell
```

To leave the environment, run:

```
exit
```

0.5 Verify the installation

Create a notebook and run:

```
import matplotlib
import numpy
import pandas
import scipy
import seaborn

print("Environment ready.")
```

If the cell runs without errors, the environment is ready for the tutorial.

0.6 Step 2: Organize your data

Your table should be long format with (at least) these columns:

- Treatment – condition label (e.g., Control, Drug)
- Replicate – biological replicate ID (e.g., R1, R2, R3)
- Value – single-cell measurement (e.g., Speed, Area, etc.)

If your measurement column is named something else (e.g., Speed), just update the code below.

0.6.1 Minimal CSV template (copy/paste to `combined.csv`)

```
Treatment,Replicate,Value
Control,R1,10.1
Control,R1,11.3
Control,R1,9.9
Control,R2,12.0
Control,R2,11.6
Drug,R1,7.2
Drug,R1,8.1
Drug,R2,9.0
Drug,R2,8.6
```

Tip: For *paired* designs (e.g., Control vs Drug measured on the same replicate lines or days), make sure the same Replicate IDs appear under each treatment.

0.7 Step 3: Load and sanity-check

```
import pandas as pd

combined = pd.read_csv("combined.csv")
assert {"Treatment", "Replicate", "Value"}.issubset(combined.columns), "Columns missing!"

print(combined.groupby(["Treatment", "Replicate"]).size())
```

- Check for typos in treatment or replicate labels.
- Ensure replicates intended to be paired have the same IDs across treatments.

0.8 Step 4: Compute replicate means (for stats & big markers)

```
ReplicateAverages = (
    combined
    .groupby(["Treatment", "Replicate"], as_index=False)["Value"]
    .mean()
    .rename(columns={"Value": "RepMean"})
)

ReplicateAverages.head()
```

0.9 Step 5: Choose and run the right statistical test

SuperPlots emphasize testing on replicate means.

- Paired t-test: same replicates appear in both treatments (e.g., R1,R2,R3 in Control *and* Drug).
- Welch's t-test (unpaired): different replicate IDs or unequal n.

```
import numpy as np
from scipy import stats
```

```
# Example: compare exactly two treatments (e.g., "Control" vs "Drug")
treatments = ["Control", "Drug"] # change if needed

pivot = ReplicateAverages.pivot_table(
    index="Replicate",
    columns="Treatment",
    values="RepMean"
)

# Detect pairing feasibility
paired_ok = all(t in pivot.columns for t in treatments) and
pivot[treatments].dropna().shape[0] ≥ 2

if paired_ok:
    df = pivot[treatments].dropna()
    stat, pvalue = stats.ttest_rel(df[treatments[0]], df[treatments[1]])
    test_used = "paired t-test (on replicate means)"
else:
    grp = [ReplicateAverages.loc[ReplicateAverages["Treatment"]==t, "RepMean"]
    for t in treatments]
    stat, pvalue = stats.ttest_ind(grp[0], grp[1], equal_var=False)
    test_used = "Welch's t-test (on replicate means)"

print(test_used, "p =", pvalue)
```

(Optional) Effect size on replicate means:

```
def cohens_d(a, b):
    a, b = np.asarray(a), np.asarray(b)
    na, nb = len(a), len(b)
    sa, sb = np.var(a, ddof=1), np.var(b, ddof=1)
    sp = np.sqrt(((na-1)*sa + (nb-1)*sb) / (na+nb-2))
    return (np.mean(b) - np.mean(a)) / sp

if paired_ok:
    # For paired designs, report mean of differences as well
```

```
diff = df[treatments[1]] - df[treatments[0]]
print("Mean of paired differences =", diff.mean())
else:
    d = cohens_d(grp[0], grp[1])
    print("Cohen's d =", d)
```

0.10 Step 6: Make the SuperPlot

Small points = single cells; large outlined points = replicate means (colored by replicate). Optional: connect paired replicate means with lines.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid", context="talk")

fig, ax = plt.subplots(figsize=(7,6))

# 1) Single-cell swarm (light alpha so replicates stand out later)
sns.swarmplot(
    data=combined, x="Treatment", y="Value",
    hue=None, dodge=False, size=3, alpha=0.6, ax=ax
)

# 2) Overlay replicate means as big, outlined markers, colored by replicate
# (We plot them on top; keep legend manageable)
mean_ax = sns.swarmplot(
    data=ReplicateAverages, x="Treatment", y="RepMean",
    hue="Replicate", dodge=True, size=12, edgecolor="k", linewidth=1.5, ax=ax
)

# Clean up legends (keep only replicate legend)
handles, labels = mean_ax.get_legend_handles_labels()
if handles:
    ax.legend(handles, labels, title="Replicate", frameon=True, loc="best")
else:
```



```

ax.legend_.remove()

# 3) Optional: connect paired replicate means across treatments
if paired_ok:
    # x positions for treatments in order they appear on the x-axis:
    xpos = {t:i for i,t in
enumerate(sorted(ReplicateAverages["Treatment"].unique()))}
    for rep, sub in ReplicateAverages.groupby("Replicate"):
        if set(treatments).issubset(set(sub["Treatment"])):
            x1, y1 = xpos[treatments[0]],
float(sub.loc[sub["Treatment"]==treatments[0], "RepMean"])
            x2, y2 = xpos[treatments[1]],
float(sub.loc[sub["Treatment"]==treatments[1], "RepMean"])
            ax.plot([x1, x2], [y1, y2], lw=1.25, alpha=0.85)

# Labels & title
ax.set_xlabel("Treatment")
ax.set_ylabel("Value")
ax.set_title("SuperPlot: single cells + replicate means")

# 4) Annotate p-value above the two groups (simple placement)
ymax = max(combined["Value"].max(), ReplicateAverages["RepMean"].max())
line_y = ymax + 0.05*(ymax if ymax != 0 else 1)
ax.plot([0, 0, 1, 1], [line_y, line_y*1.02, line_y*1.02, line_y], lw=1.5, c="k")
ax.text(0.5, line_y*1.04, f"p = {pvalue:.3g}\n{test_used}", ha="center",
va="bottom")

plt.tight_layout()
plt.show()

```

Color consistency across treatments: using `hue="Replicate"` ensures the same replicate keeps the same color in all conditions.

0.11 Step 7: Save publication-ready figures

```
plt.savefig("superplot.png", dpi=300, bbox_inches="tight")
plt.savefig("superplot.svg", bbox_inches="tight") # vector
```

0.12 Step 8: Wrap it into a reusable function

Drop this into a script/notebook and call `make_superplot(...)` for different datasets.

```
def make_superplot(
    df, value_col="Value", treatment_col="Treatment", replicate_col="Replicate",
    treatments=None, connect_paired=True, outfile=None, figsize=(7,6)
):
    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    import seaborn as sns
    from scipy import stats

    if treatments is None:
        treatments = list(df[treatment_col].unique())
        if len(treatments) != 2:
            raise ValueError("This helper currently supports exactly two
                               treatments. Provide `treatments=[A,B]`.")

    # Replicate means
    rep_means = (
        df.groupby([treatment_col, replicate_col], as_index=False)[value_col]
        .mean()
        .rename(columns={value_col: "RepMean"})
    )

    # Stats on replicate means
    pivot = rep_means.pivot_table(index=replicate_col, columns=treatment_col,
                                   values="RepMean")
    paired_ok = all(t in pivot.columns for t in treatments) and
    pivot[treatments].dropna().shape[0] >= 2
```

```
if paired_ok:
    sub = pivot[treatments].dropna()
    stat, pvalue = stats.ttest_rel(sub[treatments[0]], sub[treatments[1]])
    test_used = "paired t-test (replicate means)"
else:
    g0 = rep_means.loc[rep_means[treatment_col]==treatments[0], "RepMean"]
    g1 = rep_means.loc[rep_means[treatment_col]==treatments[1], "RepMean"]
    stat, pvalue = stats.ttest_ind(g0, g1, equal_var=False)
    test_used = "Welch's t-test (replicate means)"

sns.set(style="whitegrid", context="talk")
fig, ax = plt.subplots(figsize=figsize)

sns.swarmplot(data=df, x=treatment_col, y=value_col, hue=None, size=3,
alpha=0.6, ax=ax)
mean_ax = sns.swarmplot(
    data=rep_means, x=treatment_col, y="RepMean",
    hue=replicate_col, dodge=True, size=12, edgecolor="k", linewidth=1.5,
ax=ax
)

handles, labels = mean_ax.get_legend_handles_labels()
if handles:
    ax.legend(handles, labels, title="Replicate", frameon=True, loc="best")
else:
    ax.legend_.remove()

if connect_paired and paired_ok:
    xpos = {t:i for i,t in
enumerate(sorted(rep_means[treatment_col].unique()))}
    for rep, sub in rep_means.groupby(replicate_col):
        if set(treatments).issubset(set(sub[treatment_col])):
            x1 = xpos[treatments[0]]
            x2 = xpos[treatments[1]]
            y1 = float(sub.loc[sub[treatment_col]==treatments[0], "RepMean"])
            y2 = float(sub.loc[sub[treatment_col]==treatments[1], "RepMean"])
```

```

        ax.plot([x1, x2], [y1, y2], lw=1.25, alpha=0.85)

    ax.set_xlabel("Treatment")
    ax.set_ylabel(value_col)
    ax.set_title("SuperPlot: single cells + replicate means")

    ymax = max(df[value_col].max(), rep_means["RepMean"].max())
    line_y = ymax + 0.05*(ymax if ymax != 0 else 1)
    ax.plot([0, 0, 1, 1], [line_y, line_y*1.02, line_y*1.02, line_y], lw=1.5,
c="k")
    ax.text(0.5, line_y*1.04, f"p = {pvalue:.3g}\n{test_used}", ha="center",
va="bottom")

plt.tight_layout()
if outfile:
    plt.savefig(outfile, dpi=300, bbox_inches="tight")
return fig, ax, {"pvalue": float(pvalue), "test": test_used,
"replicate_means": rep_means}

```

Usage:

```

fig, ax, stats_info = make_superplot(
    combined,
    value_col="Value",
    treatment_col="Treatment",
    replicate_col="Replicate",
    treatments=["Control", "Drug"],
    connect_paired=True,
    outfile="superplot.png")
print(stats_info)

```

0.13 Step 9: Common variants

- Different metric name: Change value_col to your column (e.g., "Speed").
- Log scale: ax.set_yscale("log") (after plotting).
- More than two treatments: Still show all as swarms + replicate means; for stats, compare

pairs (e.g., Dunnett vs control) by looping treatments and correcting for multiple testing (e.g., Benjamini–Hochberg).

0.14 Step 10: Reproducibility tips

- Keep raw single-cell data and the replicate-mean table you used for stats.
- Fix random states where relevant (not needed for `swarmplot`, but good practice elsewhere).
- Export both figure (.png, .svg) and the `ReplicateAverages` table for the Methods/Source Data.

0.14.1 FAQ

- Why test on replicate means? Because replicates are the unit of biological independence. Testing on thousands of single cells would inflate n and understate uncertainty.
- Paired or unpaired? If the same biological replicate appears across conditions, use paired; otherwise, use Welch's (unpaired).